TV Studio: пошаговый гайд — большое приложение с окнами и диалогами

Nov 12, 2026 · @Andrey

Что строим

За 8 шагов собираем в TV Studio программу «Склад»: 4 немодальных окна, 6 диалогов, меню, строку состояния и около 15 команд. Окна и диалоги рисуются в дизайнере, меню и команды описываются в модели приложения, а поведение пишется в `src/main.rs`. Каждый шаг заканчивается сборкой (Ctrl+F9), поэтому программа всё время работает.

Что понадобится

- Rust и `cargo` в `PATH` (или путь к нему в «Настройки → Параметры среды»).
- tvision-rs: из git или локальной копии рядом с tvstudio (`work/tvision-rs`, `work/tvstudio`).
- Для подсказок в редакторе — `rustup component add rust-analyzer` и `cargo install --locked crates-lsp`. Без них всё работает, только подсказки берутся из словаря проекта.

Формы программы «Склад»

Форма (`forms::имя`)	Класс	Заголовок	Главное на форме	Откуда открывается
`goods`	Window, №1	Товары	таблица товаров, кнопки Добавить / Изменить / Удалить	меню «Справочники», F3
`good_edit`	Dialog	Товар	строки ввода с проверками, поле со списком, переключатели, флажки, многострочное поле	кнопки окна «Товары»
`suppliers`	Window, №2	Поставщики	таблица поставщиков	меню «Справочники»
`supplier_edit`	Dialog	Поставщик	строки ввода, телефон с маской	кнопки окна «Поставщики»

Форма (`forms::имя`)	Класс	Заголовок	Главное на форме	Откуда открывается
`categories`	Window, №3	Категории	дерево категорий	меню «Справочники»
`find`	Dialog	Поиск	строка ввода с историей, флажки условий	меню «Правка», Ctrl+F
`settings`	Dialog	Настройки	переключатели, флажки, линии-разделители	меню «Сервис»
`journal`	Window, №4	Журнал	многострочное поле с прокруткой	меню «Окно»
`export`	Dialog	Экспорт	полоса выполнения	меню «Файл»
`about`	Dialog	О программе	текст и кнопка OK (создаётся вместе с проектом)	меню «Справка»

Как устроен проект. Модель (`sklad.tvsp`) и формы (`forms/*.tvf`) — текстовые файлы TOML, их можно хранить в git. Перед каждой сборкой студия по ним заново пишет `src/ui.rs` и `src/forms/*.rs` . Эти файлы руками не правят. Файл `src/main.rs` создаётся один раз и дальше принадлежит вам.

Файл	Кто пишет	Что в нём
`sklad.tvsp`	студия (модель)	меню, строка состояния, команды, параметры
`forms/имя.tvf`	студия (дизайнер)	элементы формы и их свойства
`src/ui.rs`	генератор	`cmd::*` , `menu_bar()` , `status_line()` , `handle_command()` , `setup()` , `startup()`
`src/forms/имя.rs`	генератор	`ИмяData` , `ИмяIds` , `build()` , а также `run()` у диалога или `open()` / `read()` у окна
`src/main.rs`	вы	данные программы и реакция на команды

Шаг 1. Создаём проект

Итог шага: пустая программа с меню и диалогом «О программе», которая собирается и запускается.

1. Запустите студию: `cargo run --release` в каталоге tvstudio.
2. **Файл → Новый проект.** В диалоге выберите тип **«Приложение на tvision-rs»**. Тип «Программа на Rust» даёт проект без форм и без Инспектора, нам он не подходит.
3. Заполните поля:
   - **Имя** — `sklad`. Это имя пакета Cargo, поэтому только латиница, цифры и `_`.
   - **Каталог** — где создать папку проекта.
   - **tvision-rs** — ссылка git `https://github.com/cat386/tvision-rs` или путь к локальной копии. Относительный путь считается от каталога проекта. Для большого проекта удобнее локальная копия: сборка не зависит от сети.
4. Студия создаст `Cargo.toml`, `src/main.rs`, `sklad.tvsp`, форму `about` и сразу сгенерирует код. Форма «О программе» откроется в дизайнере.
5. Нажмите **Ctrl+F9**. Первая сборка с tvision-rs займёт немного времени, вывод cargo идёт в окно «Сообщения». Программа займёт экран: проверьте «Файл → О программе», выйдите по Alt+X и нажмите Enter, чтобы вернуться в студию.

Раскладка окон студии

Где	Окно	Зачем
слева	дизайнеры форм, редакторы кода	рабочее место
справа сверху	Проект	дерево: формы, модель, исходники, Cargo.toml; Enter открывает узел
справа снизу	Инспектор	сверху список компонентов, под ним свойства; F11 — к свойствам и обратно
внизу	Сообщения	вывод cargo, результаты поиска, события среды

Если окна разъехались, **Окно → Раскладка по умолчанию** вернёт их на место.

Совет: сразу сделайте `git init` и первый коммит. Модель и формы — текст TOML, поэтому в истории будет видно, какие именно свойства менялись.

Шаг 2. Модель приложения: команды, меню, строка состояния

Итог шага: полное меню «Склада» и все команды описаны до того, как нарисована первая форма. В большом приложении так проще: кнопкам форм нужны уже существующие команды, иначе проверка перед сборкой сообщит «у кнопки неизвестная команда».

2.1. Команды (Проект → Команды)

Команда — это константа `ui::cmd::ИМЯ` с номером и действием. Номера 0–99 заняты стандартными `cm::*`. Команды с номерами 100–255 можно запрещать (`app.disable_command`), команды старше 255 разрешены всегда. Поэтому командам, которые будут прятаться («Изменить» и «Удалить» при пустой таблице), давайте номера меньше 256.

У команды три варианта действия:

- **ваш код** — команду обрабатывает `main.rs`;
- **открыть форму** — диалог показывается через `run()`, окно открывается через `open()` с данными по умолчанию;
- **сообщение** — окно с текстом.

«Открыть форму» подходит, только когда форме не нужны данные программы. Окну «Товары» нужна таблица товаров, а диалогу «Настройки» — текущие настройки, поэтому у них действие «ваш код». Действие можно задать сразу, даже если формы ещё нет. В `ui.rs` тогда появится комментарий «формы нет в проекте», а сама сборка пройдёт.

Команда	Номер	Действие	Зачем
`GOODS`	100	ваш код	открыть окно «Товары» с данными
`SUPPLIERS`	101	ваш код	окно «Поставщики»
`CATEGORIES`	102	открыть форму `categories`	дерево категорий задано прямо на форме
`JOURNAL`	103	ваш код	журнал с текущим текстом
`FIND`	104	ваш код	диалог «Поиск» и переход к найденному
`SETTINGS`	105	ваш код	диалог «Настройки»

Команда	Номер	Действие	Зачем
`EXPORT`	106	ваш код	диалог с полосой выполнения
`GOOD_ADD`, `GOOD_EDIT`, `GOOD_DEL`	110–112	ваш код	кнопки окна «Товары»
`SUP_ADD`, `SUP_EDIT`, `SUP_DEL`	113–115	ваш код	кнопки окна «Поставщики»
`ABOUT`	уже есть	открыть форму `about`	создана вместе с проектом

Свободные номера между группами (107–109) оставлены на будущее. Редактор не даст занять номер стандартной команды. В поле комментария пишите, что делает команда: он попадёт в `ui.rs` как `///`.

2.2. Строка меню (Проект → Строка меню)

В редакторе меню есть кнопки «Пункт», «Подменю» и «Разделитель». Ctrl+← и Ctrl+→ меняют уровень вложенности, Ctrl+↑ и Ctrl+↓ — порядок. Тильда отмечает горячую букву: `~Т~овары`. Клавиши записываются как `F3`, `Ctrl+F`, `Alt+X`, `Shift+F6`. Если в пункте написать имя новой команды, она сразу появится в списке команд.

```
~Ф~айл
    ~Э~кспорт...            EXPORT
    ──────
    В~ы~ход          Alt+X   cm::QUIT
~П~равка
    ~Н~айти...       Ctrl+F  FIND
~С~правочники
    ~Т~овары         F3      GOODS
    ~П~оставщики             SUPPLIERS
    ~К~атегории               CATEGORIES
С~е~рвис
    ~Н~астройки...             SETTINGS
~О~кно
    ~Ж~урнал                  JOURNAL
    ──────
    ~С~ледующее     F6      cm::NEXT
    ~З~акрыть       Alt+F3  cm::CLOSE
Спр~а~вка
    ~О~ программе              ABOUT
```

У каждого пункта можно задать номер справки (help context). Это пригодится, если позже добавить справку по F1.

2.3. Строка состояния (Проект → Строка состояния)

Текст	Клавиша	Команда
`~Alt+X~ Выход`	Alt+X	`cm::QUIT`
`~F3~ Товары`	F3	`GOODS`
`~Ctrl+F~ Поиск`	Ctrl+F	`FIND`
`~F10~ Меню`	F10	`cm::MENU`

Пункт без клавиши показывается как простая надпись-подсказка.

2.4. Параметры программы (Проект → Параметры программы)

- **Заголовок** — «Склад».
- **Русские надписи tvision-rs** — включите. Тогда в `ui::setup()` появится `locale::set_language(locale::Language::Ru)`, и стандартные диалоги будут по-русски (Да / Нет / Отмена, выбор файла).
- **Стартовая форма** — пока пусто. Окно «Товары» мы откроем при запуске из своего кода (шаг 6), потому что ему нужны данные.

Проверка: Ctrl+F9. Меню уже полное, пункты с действием «ваш код» пока ничего не делают.

Шаг 3. Первая форма-диалог «Товар»

Итог шага: модальный диалог `good_edit` с восемью полями, проверками ввода и правильным порядком Tab. Это самая подробная часть гайда: остальные формы делаются теми же приёмами.

3.1. Создать форму

1. **Форма → Новая форма.**
2. В диалоге укажите:
   - Имя: `good_edit`. Это имя модуля `forms::good_edit`, из него же получаются имена типов `GoodEditData` и `GoodEditIds`.
   - Заголовок: «Товар».
   - Ширина 60, высота 20.

- Класс: **Dialog**.

3. Нажмите OK. Кнопки OK (по умолчанию, `cm::OK`) и «Отмена» (`cm::CANCEL`) студия ставит у нижнего края сама.

Окно дизайнера — это и есть будущий диалог: та же рамка, цвет и размер, внутри настоящие виджеты tvision-rs. Размер формы меняется перетаскиванием за правый нижний угол, положение — за заголовок.

3.2. Расставить элементы

Как добавить элемент:

1. Щёлкните по пустому месту формы. Так задаётся точка вставки.
2. В Инспекторе, в списке компонентов, выберите нужный и нажмите Enter (или сделайте двойной щелчок).

Другие способы: Ins в дизайнере или «Форма → Добавить элемент».

Порядок добавления важен: порядок элементов — это порядок перехода по Tab и порядок полей в `GoodEditData`. Добавляйте сверху вниз. Исправить порядок можно потом (см. 3.4).

Имя	Компонент	X, Y, ширина × высота	Свойства	Метка
`name`	Строка ввода	16, 2, 40 × 1	Длина 80	`~Н~аименование:`
`sku`	Строка ввода	16, 4, 12 × 1	Проверка `picture:###-####`	`~А~ртикул:`
`category`	Поле со списком	16, 6, 30 × 1	Пункты `Электроника | Химия | Продукты | Канцтовары`	`~К~атегория:`
`qty`	Строка ввода	16, 8, 10 × 1	Проверка `range:0..100000`	`~К~ол-во:`
`price`	Строка ввода	40, 8, 14 × 1	Проверка `filter:0123456789.`	`~Ц~ена:`
`unit`	Переключатели	3, 11, 14 × 3	Пункты `шт | кг | л`	`~Е~д. изм.`

Имя	Компонент	X, Y, ширина × высота	Свойства	Метка
`flags`	Флажки	20, 11, 24 × 3	Пункты `Под заказ | Скидка | Снят с продажи`	—
`note`	Многострочное	16, 14, 40 × 2	Прокрутка: да	`~П~римечание:`

Удобный приём для больших форм: сделайте одну строку ввода с меткой, выделите обе (Ctrl+щелчок) и нажмите **Ctrl+D**, чтобы дублировать. Останется поменять имя и надпись. Ctrl+C / Ctrl+V переносят элементы и между формами.

Метки. Выделите поле и добавьте компонент «Метка»: она сразу свяжется с этим полем (свойство «Связь»). В работающей программе Alt+горячая буква метки переводит курсор в поле. Следите, чтобы горячие буквы в одной форме не повторялись (в таблице выше «К» дважды — у «Кол-во» стоит взять «о»: `К~о~л-во:`).

3.3. Свойства в Инспекторе

F11 переводит фокус к свойствам выделенного элемента, повторное F11 возвращает в дизайнер. Enter или F2 начинают правку, Enter применяет, Esc отменяет, пробел переключает «да / нет». Неверное значение Инспектор не примет. Все свойства сразу, включая многострочный текст, открывает диалог свойств: Enter в дизайнере, Ctrl+F2 или двойной щелчок.

Свойство	Как вводить	Пример
Имя	идентификатор Rust; станет полем `Data` и `Ids`	`price`
Надпись / Текст	`~X~` — горячая буква, `\n` — новая строка, `\x03` в начале — по центру	`~Ц~ена:`
Пункты	через `|`	`шт | кг | л`
Проверка	`picture:` маска, `range:` диапазон, `filter:` допустимые символы, `lookup:` список	`picture:###-####`
История	номер списка истории, 0 — без истории	`1`

Свойство	Как вводить	Пример
Отмечены / Выбран	флажки — битовая маска, переключатели и список — номер с 0	`0`
Команда (кнопка)	`cm::OK`, `cm::CANCEL` … или имя своей команды	`GOOD_ADD`
Недоступен	элемент серый и не принимает фокус	нет
Справка	номер контекста справки	`0`

Если ничего не выделено (Esc в дизайнере), Инспектор показывает свойства формы: Имя, Заголовок, Класс, размеры, По центру, Цвет (серый / синий / бирюзовый), Справка.

3.4. Выравнивание и порядок Tab

- Выделите несколько элементов рамкой или Ctrl+щелчком и выберите **Форма → Выровнять**. **Форма → Сетка** показывает точки привязки.
- Стрелки сдвигают выделенное на клетку, Ctrl+стрелки — на 4 клетки, Shift+стрелки меняют размер.
- **Форма → Номера Tab** показывает порядок прямо на форме. Ctrl+PgUp / Ctrl+PgDn сдвигают элемент раньше или позже, в меню есть также «В начало» и «В конец порядка».
- Ошибся — Ctrl+Z (отмена без ограничения шагов), Ctrl+Y — повтор.

3.5. Проверить и сохранить

1. **Shift+F9** (Форма → Проверить форму) показывает форму как настоящий диалог без сборки. Пройдитесь по Tab, введите артикул не по маске и убедитесь, что проверка работает. После закрытия студия покажет, какой командой закрыт диалог и какие данные он вернул.
2. **F2** сохраняет форму в `forms/good_edit.tvf`.
3. **F12** открывает сгенерированный `src/forms/good_edit.rs`. В нём появится:

```
pub struct GoodEditData {
    pub name: String,      // строка ввода
    pub sku: String,
    pub category: String,  // поле со списком
    pub qty: String,
    pub price: String,
    pub unit: u32,         // переключатели: номер выбранного
    pub flags: u32,        // флажки: битовая маска
    pub note: String,      // многострочное поле
}
pub fn build() -> (Dialog, GoodEditIds) { /* … */ }
pub fn run(app: &mut Program, data: &mut GoodEditData) -> u16 { /* … */ }
```

Метки, тексты, кнопки и линии данных не имеют, в `Data` их нет, зато они есть в `Ids`. Типы полей: строка ввода, поле со списком и многострочное поле — `String`; флажки и переключатели — `u32`; список, полоса выполнения и таблица — `i64`. F12 ещё раз вернёт вас в дизайнер.

По той же схеме сделайте ещё три диалога:

- `supplier_edit` «Поставщик»: `name`, `phone` (`picture:+#(###)###-##-##`), `email`, `city` (строка ввода с историей 2).
- `find` «Поиск», 50 × 10: `text` (история 1 — стрелка вниз покажет прошлые запросы) и флажки `opts` «Учитывать регистр | Только в наличии».
- `export` «Экспорт», 50 × 8: текст `\x03Выгрузка товаров…`, полоса выполнения `bar` (Максимум 100) и только кнопка «Отмена» (лишнюю OK удалите клавишей Del).

Шаг 4. Формы-окна (немодальные)

Итог шага: четыре окна — «Товары», «Поставщики», «Категории» и «Журнал». Они могут быть открыты одновременно и переключаются по Alt+1…Alt+4.

Диалог или окно?

	Dialog	Window
Как показывается	модально: программа ждёт закрытия	на рабочем столе, рядом с другими окнами
Функции в `forms::имя`	`build()`, `run(app, &mut data) -> u16`	`build()`, `open(app, &data) -> ViewId`, `read(app, id) -> Option<Data>`

	Dialog	Window
Готовые кнопки	OK и «Отмена»	«Закрыть» (`cm::CLOSE`)
Куда идут команды своих кнопок	в `app.modal_hook` (см. шаг 6)	в обработчик программы `App::handle_event`
Дополнительные свойства формы	—	«Номер окна» (0–9, Alt+номер), «Растягивается» (значки размера и распахивания)

Правило: если пользователь что-то вводит и нажимает OK, нужен диалог. Если он с чем-то работает долго и параллельно с другим (список, журнал, справочник), нужно окно. Класс можно поменять и позже, свойством «Класс» в Инспекторе (пробел переключает). Тогда переделайте вызовы в `main.rs`: `run` ↔ `open` / `read`.

4.1. Окно «Товары»

1. **Форма → Новая форма**: имя `goods`, заголовок «Товары», 76 × 20, класс **Window**. Кнопка «Закрыть» появится сама.
2. В свойствах формы (Esc, затем F11): «Номер окна» 1, «Растягивается» — да, «Цвет» — синий.
3. Добавьте таблицу `table` (X 2, Y 1, 72 × 14). Её настройка описана в шаге 5.
4. Добавьте линию `sep` на Y 16 с «До рамки» = да. Она отделит кнопки от таблицы и врастёт в рамку окна (`╟──╢`).
5. Добавьте кнопки в строку Y 17 и поставьте готовую кнопку `close` последней:

Имя	Надпись	Команда	По умолчанию
`add`	`~Д~обавить`	`GOOD_ADD`	нет
`edit`	`~И~зменить`	`GOOD_EDIT`	да
`del`	`~У~далить`	`GOOD_DEL`	нет
`close`	`~З~акрыть`	`cm::CLOSE`	нет

6. Shift+F9 — проверка, F2 — сохранить.

4.2. Остальные окна

Форма	Размер	Номер	Содержимое
`suppliers` «Поставщики»	70 × 18	2	таблица `table`, линия, кнопки `SUP_ADD` / `SUP_EDIT` / `SUP_DEL` / `cm::CLOSE`
`categories` «Категории»	40 × 18	3	дерево `tree` и кнопка «Закрыть»
`journal` «Журнал»	70 × 14	4	многострочное поле `log` во всё окно с прокруткой, кнопка «Закрыть»

Номера окон в пределах программы не должны повторяться, иначе Alt+номер выберет не то окно. Окнам, которых больше девяти или которые открываются во множестве копий, ставьте номер 0.

У команды `CATEGORIES` действие «открыть форму categories» (шаг 2). Теперь, когда форма есть, `ui.rs` вызовет для неё `forms::categories::open(...)`, и пункт меню заработает после сборки без единой строки своего кода.

Шаг 5. Сложные компоненты: таблица, дерево, списки, линии

Итог шага: настроены таблицы товаров и поставщиков, дерево категорий и диалог «Настройки» с разделителями. Строки, которые вы вводите в дизайнере, нужны только для образца. Настоящие данные программа подставит в шаге 6.

5.1. Таблица (Table)

Свойство	Что задаёт	Для `goods.table`
Столбцы	заголовки и ширина через двоеточие, разделитель `|`	`Наименование:28 | Артикул:10 | Кол-во:8 | Цена:10 | Ед.:6`
Строки	по строке на запись, ячейки через `|` (удобнее в диалоге свойств, Ctrl+F2)	`Чайник | 123-4567 | 12 | 2490 | шт`
Заголовки	показывать строку заголовков	да
Сетка	линии между ячейками	да
Номера строк	столбец с номерами слева	нет

Свойство	Что задаёт	Для `goods.table`
Правка	разрешить менять ячейки прямо в таблице (F2 или набор текста)	нет: товар правится в диалоге
Прокрутка	полосы прокрутки	да
Строка	строка с курсором при открытии; это и есть данные таблицы (`i64`)	0

В `GoodsData` у таблицы одно поле, `table: i64`, — номер строки под курсором. По нему программа узнаёт, какой товар изменить или удалить. Содержимое таблицы задаётся из кода через `Table::set_model` (шаг 6).

5.2. Дерево (TreeView)

В Инспекторе узлы вводятся одной строкой, уровень вложенности обозначается знаками `>`:

```
Все товары | >Электроника | >>Телефоны | >>Ноутбуки | >Хозтовары | >>Бытовая химия | >Продукты
```

В диалоге свойств то же самое пишется по строке на узел, а вложенность обозначается двумя пробелами отступа. Ещё три свойства: «Раскрыто» (ветви открыты сразу), «Линии» (рисовать линии ветвей) и «Флажки» (у каждого узла флажок с тремя состояниями). Данных в структуре формы у дерева нет. Если дерево нужно менять из программы, используйте `build()` и `CategoriesIds::tree`.

5.3. Списки, поле со списком, полосы

- **Список (ListBox)**: «Пункты» через `|`, «Столбцы» (пункты раскладываются в несколько колонок), «Прокрутка». Данные — номер выбранного пункта, `i64`.
- **Поле со списком (ComboBox)**: строка ввода и выпадающий список «Пункты». Данные — введённый текст, `String`.
- **Полоса выполнения (ProgressBar)**: «Значение» и «Максимум». Данные — текущее значение, `i64`.
- **Полоса прокрутки (ScrollBar)** — отдельная полоса, если нужна своя. У списка, таблицы, дерева и многострочного поля полосы включаются их свойством «Прокрутка».

5.4. Линии-разделители и диалог «Настройки»

Линия (Line) — горизонтальный `─` или вертикальный `│` разделитель.

- «Вертикальная» — при переключении ширина и высота меняются местами.
- «Двойная» — `═` или `║`.
- «До рамки» — линия во всю ширину или высоту окна, концы врастают в рамку (`╟┤`).

Пересекающиеся линии соединяются сами (`┼`). Данных у линии нет.

Диалог `settings` «Настройки», 56 × 16, класс Dialog:

Имя	Компонент	Где	Свойства
`theme`	Переключатели	3, 2, 20 × 3	`~С~иняя | С~е~рая | ~Б~ирюзовая`
`vsep`	Линия	X 26, Y 1, высота 8	Вертикальная: да
`currency`	Список	29, 2, 24 × 4	`RUB | USD | EUR | CNY`
`hsep`	Линия	Y 9	До рамки: да, Двойная: нет
`opts`	Флажки	3, 10, 40 × 3	`~А~втосохранение | ~П~одтверждать удаление | Вести ~ж~урнал`
`ok`, `cancel`	Кнопки	внизу	созданы автоматически

Вертикальная линия `vsep` доходит до горизонтальной `hsep`, и на стыке появится `┴`. Метки «~Т~ема» и «~В~алюта» ставятся над переключателями и списком. Проверьте результат через Shift+F9.

Шаг 6. Логика: связываем формы с данными в main.rs

Итог шага: работающая программа. Всё поведение пишется в `src/main.rs` (откройте его из дерева проекта). Сгенерированные файлы не трогаем.

6.1. Как идут команды

1. Пользователь выбирает пункт меню, нажимает клавишу строки состояния или кнопку в окне-форме.
2. Команда, которую не обработали сами окна, приходит в `App::handle_event`.
3. Сначала её пробует `ui::handle_command`: там действия из модели («открыть форму», «сообщение»).

4. Всё с действием «ваш код» обрабатывает ваш `match`.

Исключение — модальный диалог. Пока он открыт, `handle_event` не вызывается. Стандартные OK / Отмена / Да / Нет закрывают диалог сами, а свои команды кнопок внутри диалога ловятся в `app.modal_hook` (см. 6.6).

6.2. Данные и состояние программы

Шаблон `main.rs` содержит пустой `struct App;`. Для большого приложения положите в него все данные и id открытых окон:

```
mod forms;
mod ui;

use tvision::*;
use ui::cmd;

#[derive(Clone, Default)]
struct Good {
    name: String,
    sku: String,
    category: String,
    qty: i64,
    price: f64,
    unit: u32,  // 0 — шт, 1 — кг, 2 — л
    flags: u32, // бит 0 — под заказ, 1 — скидка, 2 — снят
    note: String,
}

const UNITS: [&str; 3] = ["шт", "кг", "л"];

impl Good {
    /// Товар → данные диалога «Товар».
    fn to_form(&self) -> forms::good_edit::GoodEditData {
        forms::good_edit::GoodEditData {
            name: self.name.clone(),
            sku: self.sku.clone(),
            category: self.category.clone(),
            qty: self.qty.to_string(),
            price: self.price.to_string(),
            unit: self.unit,
            flags: self.flags,
            note: self.note.clone(),
        }
    }
```

```
    /// Данные диалога → товар (с проверкой того, чего не сделали
валидаторы).
    fn from_form(d: &forms::good_edit::GoodEditData) -> Result<Good, String>
{
        if d.name.trim().is_empty() {
            return Err("Укажите наименование.".into());
        }
        Ok(Good {
            name: d.name.trim().into(),
            sku: d.sku.clone(),
            category: d.category.clone(),
            qty: d.qty.trim().parse().unwrap_or(0),
            price: d.price.trim().parse().map_err(|_| "Цена —
число.".to_string())?,
            unit: d.unit,
            flags: d.flags,
            note: d.note.clone(),
        })
    }

    /// Ячейки строки таблицы «Товары».
    fn cells(&self) -> Vec<String> {
        vec![
            self.name.clone(),
            self.sku.clone(),
            self.qty.to_string(),
            format!("{:.2}", self.price),
            UNITS.get(self.unit as usize).unwrap_or(&"?").to_string(),
        ]
    }
}

#[derive(Default)]
struct App {
    goods: Vec<Good>,
    settings: forms::settings::SettingsData,
    find: forms::find::FindData,
    log: String,
    /// Окно «Товары»: (id окна, id таблицы), если открыто.
    goods_win: Option<(ViewId, ViewId)>,
}
```

Идея простая: у каждой сущности есть `to_form` и `from_form`, которые переводят её в `ИмяData` и обратно. Если в дизайнере добавить или переименовать поле, компилятор укажет ровно те места, которые нужно поправить.

6.3. Окно с данными: build() вместо open()

`forms::goods::open()` открывает окно с таблицей-образцом из дизайнера. Чтобы показать настоящие товары, берём `build()`, меняем таблицу по id из `GoodsIds` и сами вставляем окно:

```
/// Столбцы таблицы товаров (те же, что в дизайнере).
const GOODS_COLS: [(&str, i32); 5] = [("Наименование", 28), ("Артикул", 10),
("Кол-во", 8), ("Цена", 10), ("Ед.", 6)];

fn fill_goods(t: &mut Table, goods: &[Good]) {
    let (row, _) = t.cursor();
    let heads: Vec<&str> = GOODS_COLS.iter().map(|c| c.0).collect();
    let rows = goods.iter().map(Good::cells).collect();
    t.set_model(Box::new(VecTableModel::new(&heads, rows)));
    // set_model сбрасывает столбцы и курсор — возвращаем свои.
    t.set_columns(GOODS_COLS.iter().map(|(n, w)|
TableColumn::new(*w).title(n)).collect());
    if !goods.is_empty() {
        t.set_cursor(row.min(goods.len() - 1), 0);
    }
}

impl App {
    fn open_goods(&mut self, app: &mut Program) {
        // Уже открыто — просто перейти в него.
        if let Some((win, _)) = self.goods_win {
            if app.find::<Window>(win).is_some() {
                app.focus(win);
                return;
            }
        }
        let (mut w, ids) = forms::goods::build();
        if let Some(t) = w.group.find_mut::<Table>(ids.table) {
            fill_goods(t, &self.goods);
        }
        let win = app.insert_window(w);
        self.goods_win = Some((win, ids.table));
    }

    /// Перерисовать таблицу, если окно открыто.
    fn refresh_goods(&self, app: &mut Program) {
        if let Some((_, t)) = self.goods_win {
            if let Some(t) = app.find_mut::<Table>(t) {
                fill_goods(t, &self.goods);
            }
```

```
        }
    }

    /// Номер товара под курсором таблицы.
    fn current_good(&self, app: &Program) -> Option<usize> {
        let (_, t) = self.goods_win?;
        let (row, _) = app.find::<Table>(t)?.cursor();
        (row < self.goods.len()).then_some(row)
    }
}
```

Ширины столбцов теперь заданы и в дизайнере, и в `GOODS_COLS`. Меняя одно, меняйте и другое.

6.4. Диалоги: run() и разбор результата

`run()` заносит `data` в диалог, ждёт закрытия и возвращает команду. Если диалог закрыт не «Отменой», введённое уже лежит в `data`:

```
impl App {
    fn add_good(&mut self, app: &mut Program) {
        let mut data = forms::good_edit::GoodEditData::default();
        if forms::good_edit::run(app, &mut data) != cm::OK {
            return;
        }
        match Good::from_form(&data) {
            Ok(g) => {
                self.write_log(&format!("добавлен товар «{}»", g.name));
                self.goods.push(g);
                self.refresh_goods(app);
            }
            Err(e) => {
                message_box(app, &e, mf::ERROR | mf::OK_BUTTON);
            }
        }
    }

    fn edit_good(&mut self, app: &mut Program) {
        let Some(i) = self.current_good(app) else { return };
        let mut data = self.goods[i].to_form();
        if forms::good_edit::run(app, &mut data) == cm::OK {
            match Good::from_form(&data) {
                Ok(g) => {
                    self.goods[i] = g;
                    self.refresh_goods(app);
                }
```

```
                Err(e) => {
                    message_box(app, &e, mf::ERROR | mf::OK_BUTTON);
                }
            }
        }
    }

    fn delete_good(&mut self, app: &mut Program) {
        let Some(i) = self.current_good(app) else { return };
        let ask = format!("\x03Удалить «{}»?", self.goods[i].name);
        if message_box(app, &ask, mf::CONFIRMATION | mf::YES_BUTTON |
mf::NO_BUTTON) == cm::YES {
            let g = self.goods.remove(i);
            self.write_log(&format!("удалён товар «{}»", g.name));
            self.refresh_goods(app);
        }
    }

    fn settings(&mut self, app: &mut Program) {
        // Работаем с копией: «Отмена» ничего не меняет.
        let mut data = self.settings.clone();
        if forms::settings::run(app, &mut data) == cm::OK {
            self.settings = data;
        }
    }

    fn write_log(&mut self, s: &str) {
        self.log.push_str(s);
        self.log.push('\n');
    }
}
```

Для поиска храните `FindData` в `App`, как показано выше. Тогда диалог открывается с прошлым запросом, а история строки ввода (свойство «История») помнит и старые. Флажки читаются как биты: `self.find.opts & 1 != 0` — «Учитывать регистр».

6.5. Обработчик команд и запрет недоступного

```
impl AppHandler for App {
    fn handle_event(&mut self, app: &mut Program, ev: &mut Event) {
        let Some(command) = ev.command() else { return };
        // Действия из модели (CATEGORIES, ABOUT).
        if ui::handle_command(app, command) {
            ev.clear();
```

```
            return;
        }
        match command {
            cmd::GOODS => self.open_goods(app),
            cmd::GOOD_ADD => self.add_good(app),
            cmd::GOOD_EDIT => self.edit_good(app),
            cmd::GOOD_DEL => self.delete_good(app),
            cmd::SETTINGS => self.settings(app),
            cmd::JOURNAL => {
                let data = forms::journal::JournalData { log:
self.log.clone() };
                forms::journal::open(app, &data);
            }
            cmd::EXPORT => self.export(app),
            _ => return, // не наша команда — не трогаем
        }
        ev.clear();
    }

    /// Когда событий нет: разрешить только то, что имеет смысл.
    fn idle(&mut self, app: &mut Program) {
        let has_row = self.current_good(app).is_some();
        for c in [cmd::GOOD_EDIT, cmd::GOOD_DEL] {
            if has_row { app.enable_command(c) } else {
app.disable_command(c) }
        }
    }
}

fn main() -> std::io::Result<()> {
    ui::setup();
    let mut app = Program::with_terminal(Some(ui::menu_bar()),
Some(ui::status_line()))?;
    let mut handler = App::default(); // здесь же — чтение данных из файла
    ui::startup(&mut app);
    handler.open_goods(&mut app); // «Товары» открыты сразу
    app.run(&mut handler);
    Ok(())
}
```

Запрещённые команды рисуются серым в меню, строке состояния и на кнопках. Работает это только для номеров 100–255, поэтому в шаге 2 мы и дали кнопкам номера 110–115.

6.6. Долгая работа с полосой выполнения и свои команды в диалоге

Диалог «Экспорт» показывается через `exec_dialog_idle`. Функция-фон вызывается примерно 10 раз в секунду, а `Some(cmd)` закрывает окно. За один вызов обрабатывайте порцию, а не по одной записи. Кнопка «Отмена» работает сама.

```
impl App {
    fn export(&mut self, app: &mut Program) {
        let (d, ids) = forms::export::build();
        let total = self.goods.len();
        let mut done = 0;
        let mut out = String::new();
        let goods = &self.goods;
        let (command, _) = app.exec_dialog_idle(d, |app, _| {
            for g in goods.iter().skip(done).take(200) {
                out.push_str(&g.cells().join(";"));
                out.push('\n');
            }
            done = (done + 200).min(total);
            if let Some(bar) = app.find_mut::<ProgressBar>(ids.bar) {
                bar.set_value((done * 100 / total.max(1)) as i64);
            }
            (done >= total).then_some(cm::OK)
        });
        if command == cm::OK {
            if let Err(e) = std::fs::write("export.csv", out) {
                message_box(app, &e.to_string(), mf::ERROR | mf::OK_BUTTON);
            }
        }
    }
}
```

Если в модальном диалоге есть кнопка со своей командой (например, «Выбрать поставщика…» в «Товаре»), её событие не дойдёт до `handle_event`. Его ловит `app.modal_hook`:

```
app.modal_hook = Some(Box::new(|app, ev| {
    if ev.command() == Some(cmd::PICK_SUPPLIER) {
        // здесь можно открыть вложенный диалог
        ev.clear();
    }
}));
```

Замыкание живёт отдельно от `App`. Данные, которые ему нужны, передавайте через `Rc<RefCell<…>>` или копируйте в него.

Самостоятельно по той же схеме допишите поставщиков (`SUPPLIERS` , `SUP_ADD` …), поиск (`FIND` : `run` , затем `set_cursor` на найденную строку) и сохранение данных в файл. Готовый образец такой программы — пример `examples/guide_contacts` (записная книжка с таблицей, диалогом, запретом команд и файлами).

Шаг 7. Сборка, запуск, ошибки и подсказки

Итог шага: цикл «поправил → собрал → проверил» занимает секунды.

7.1. Сборка

Клавиша	Действие	Когда применять
Alt+F9	`cargo check`	быстрая проверка после правки кода
F9	`cargo build`	полная сборка
Ctrl+F9	собрать и запустить	проверить программу целиком
Сборка → Тесты	`cargo test`	тесты проекта
Сборка → Остановить	прервать cargo	долгая сборка не нужна

Перед каждой сборкой студия сохраняет файлы, проверяет модель и заново генерирует `ui.rs` и `forms/*.rs` . Проверка ловит повторяющиеся имена форм и команд и кнопки с неизвестной командой. cargo работает в фоне, среда не замирает. При запуске экран отдаётся программе; после её выхода нажмите Enter. Сгенерировать код без сборки можно командой **Проект → Генерировать код**.

7.2. Ошибки

- Вывод cargo идёт в «Сообщения»: `■` — ошибка, `▲` — предупреждение. Enter или двойной щелчок по строке открывает файл на нужной строке.
- **F8 / Shift+F8** — к следующей и предыдущей ошибке. В редакторе строки помечены `●` и `▲` на полосе номеров.
- Ошибка в `src/forms/*.rs` почти всегда означает, что форма и `main.rs` разошлись (поле переименовано или удалено). Исправляйте `main.rs` или форму в дизайнере, но не сгенерированный файл: он перезапишется.

7.3. Редактор кода и LSP

Пока проект открыт, работают rust-analyzer (файлы `.rs`) и crates-lsp (`Cargo.toml`). Ход индексации виден в заголовке окна сообщений.

Возможность	Как вызвать	Чем помогает в большом проекте
Подсказки при наборе	сами после двух букв; выбор стрелками, вставка Tab	имена форм, полей `Data` и команд не нужно помнить; недостающий `use` допишется сам
Дополнение	Ctrl+Space, само после `.` и `::`	`forms::` — список всех форм, `cmd::` — всех команд
К определению / назад	F12 / Alt+←	из `main.rs` в сгенерированную форму и обратно
Найти ссылки	Shift+F12	где используется команда или поле
Быстрые исправления	Alt+Enter (Ctrl+.)	«добавить use», «заполнить match»
Диагностика при наборе	сама; все замечания — Поиск → Диагностика LSP	ошибки видны до сборки
Поиск по проекту	Поиск → Найти в файлах	найти все вызовы `forms::goods`

Ещё три вещи делаются сами. При сохранении (F2) файл форматируется rustfmt, и Ctrl+Z отменяет это одним шагом. Скобки и кавычки закрываются. В `Cargo.toml` crates-lsp показывает, есть ли более новая версия зависимости. Всё это выключается в «Настройки → Параметры среды».

Шаг 8. Импорт и организация большого проекта

Когда форм и команд становится много, порядок держится на трёх вещах: соглашениях о именах, диапазонах номеров команд и разбиении `main.rs` на модули.

8.1. Уже есть программа на tvision-rs?

- **Файл → Импорт программы** (или `tvs-import ПРОГРАММА.rs КАТАЛОГ ИМЯ`). Исходники копируются, а строка меню, строка состояния и константы команд переходят в модель. Меню, собранное функциями или в цикле, остаётся в коде.
- **Проект → Формы из кода** находит в исходниках диалоги (`Dialog::new`, `Dialog::centered` и `d.add`) и делает из них формы `forms/*.tvf`. Код программы при этом не меняется. Свои виджеты и элементы внутри условий перечисляются в окне сообщений. Окна `Window` пока не извлекаются.
- Переводите программу по одной форме: замените ручную сборку диалога на `forms::имя::build()` или `run()`, соберите и проверьте, затем переходите к следующей.

8.2. Соглашения о именах и номерах

Что	Правило	Пример
Окно-список	имя сущности во множественном числе	`goods`, `suppliers`
Диалог правки	единственное число + `_edit`	`good_edit`, `supplier_edit`
Элементы формы	смысловые имена вместо `input1`, `button2`: они станут полями `Data` и `Ids`	`price`, `unit`
Команды открытия	имя окна заглавными, 100–109	`GOODS`
Команды сущности	свой блок номеров на сущность, все меньше 256	`GOOD_*` 110–112, `SUP_*` 113–115
Номера окон	1–9 только у главных окон, без повторов	«Товары» — 1
Контексты справки	свой номер у формы и важных полей (свойство «Справка»)	`good_edit` — 1100

8.3. Как разложить код

`main.rs` принадлежит вам, и рядом с ним можно заводить свои модули. Студия перезаписывает только `ui.rs` и `forms/`.

```
src/
  main.rs        main(), struct App, handle_event, idle — только раздача
команд
  goods.rs       Good, to_form/from_form, fill_goods, add/edit/delete
  suppliers.rs   то же для поставщиков
  storage.rs     чтение и запись данных, экспорт
  ui.rs          ⚙ сгенерирован
  forms/         ⚙ сгенерированы
```

В `main.rs` остаётся только `match` по командам, который вызывает методы `App` из модулей. Методы `impl App` можно объявлять в любом модуле крейта. В дереве проекта сгенерированные файлы помечены ⚙.

8.4. Советы

- **Не правьте `ui.rs` и `forms/*.rs`.** Нужно что-то, чего нет в дизайнере (свой виджет, особая настройка таблицы)? Возьмите `build()`, поменяйте диалог через `Ids` или добавьте виджет `d.add(...)` и покажите его сами (`app.exec_dialog(d)` или `app.insert_window(w)`).
- **Переименование** элемента или формы меняет имена в сгенерированном коде. Сразу запустите Alt+F9: компилятор покажет все места в вашем коде.
- **Порядок Tab** проверяйте в каждой форме (Форма → Номера Tab), а горячие буквы — на повторы через Shift+F9.
- **Один стиль для всех форм.** Метки слева шириной 14, поля с X 16, кнопки в нижней строке, диалоги серые, окна синие. Копируйте готовые блоки между формами через Ctrl+C / Ctrl+V.
- **git и ревью.** `.tvsp` и `.tvf` — TOML (`[[controls]]`, `[[app.menu]]`), поэтому разница видна построчно. Сгенерированные файлы тоже храните в git: тогда проект собирается обычным `cargo build` без студии.
- **Тесты.** Данные и `from_form` проверяются обычными `#[test]`. Интерфейс проверяется на «невидимом» терминале tvision-rs — как в `examples/guide_contacts`. Запуск: Сборка → Тесты.
- **Растёт до нескольких крейтов?** Вынесите логику в библиотеку в общем workspace. Участники `[workspace] members` видны в дереве проекта отдельными узлами ◇. F9 собирает весь workspace, Ctrl+F9 запускает участника, в котором открыт текущий файл.

Шпаргалка и чек-лист

Клавиши

Где	Клавиша	Действие
везде	F1 / F10 / Alt+X	справка / меню / выход
везде	F2 / Shift+F2	сохранить / сохранить всё
везде	F6 / Shift+F6, Alt+1…9	следующее / предыдущее окно, окно по номеру
везде	F11	к свойствам в Инспекторе и обратно
дизайнер	Ins / Del / Enter	добавить / удалить / свойства элемента
дизайнер	Стрелки, Ctrl+стрелки, Shift+стрелки	сдвиг на 1, сдвиг на 4, размер
дизайнер	Ctrl+C / X / V / D	копировать / вырезать / вставить / дублировать
дизайнер	Ctrl+Z / Ctrl+Y, Ctrl+A	отменить / повторить, выделить всё
дизайнер	Ctrl+PgUp / Ctrl+PgDn	раньше / позже в порядке Tab
дизайнер	Shift+F9 / F12	проверить форму / код формы
Инспектор	Enter, F2 / Пробел / Esc	править / да-нет / отмена
редактор	Ctrl+Space, Tab	дополнение, выбрать подсказку
редактор	F12 / Alt+← / Shift+F12	к определению / назад / ссылки
редактор	Alt+Enter, Alt+G	быстрые исправления, перейти к строке
сборка	Alt+F9 / F9 / Ctrl+F9	check / build / запуск
сборка	F8 / Shift+F8	следующая / предыдущая ошибка

Чек-лист новой формы

- [ ] Выбран класс: Dialog (ввод и OK) или Window (долгая работа)
- [ ] Имена формы и элементов смысловые, без `input1`
- [ ] У каждого поля есть метка со связью и горячей буквой без повторов
- [ ] У полей ввода заданы «Длина» и «Проверка»

- [ ] Порядок Tab идёт сверху вниз и слева направо (Форма → Номера Tab)
- [ ] У кнопок существующие команды; команды, которые нужно запрещать, имеют номера 100–255
- [ ] У окна заданы номер и «Растягивается»
- [ ] Shift+F9 пройден: данные возвращаются так, как ожидалось
- [ ] В `main.rs` есть `to_form` / `from_form` и ветка `match` для команды
- [ ] Ctrl+F9: форма открывается из меню и сохраняет данные
- [ ] Модель, форма и код закоммичены вместе

Встроенная справка студии (F1) и README репозитория cat386/tvstudio описывают то же самое короче. Два готовых примера для изучения: `examples/guide_contacts` (таблица, диалог, запрет команд, файлы) и `examples/tvdemo` (много окон и диалогов).